Granite — Full Defense Preparation Pack

F26-060-GRANITE · Proposal Defense · Stream A

Compiled from the defense-prep folder: timed speaking script, expanded per-slide notes, full CCP/spike reasoning, and the literature-review panel narrative.

Contents

- 1. Full Defense Speaking Script — 7 to 10 Minutes
- 2. Slides 5, 6, 7 — Expanded Speaking Notes
- 3. CCP, Spikes & Contribution — Full Panel-Defense Reasoning
- 4. Literature Review — Workflow & Panel Narrative

Full Defense Speaking Script — 7 to 10 Minutes

Written strictly from what's actually on each slide right now (checked against the built deck, not an old draft). Read it aloud once at a natural, unhurried pace before the real thing — this is timed to land around **8.5 minutes**, with slack built in on both ends (7 min if you move briskly, 10 min if you pause for emphasis or a slow breath). Time budgets are guides, not a stopwatch to watch mid-sentence.

Total: ~1,100 words, ~8.5 min at a normal speaking pace (~130 wpm).

Slide 1 — Title (~20s)

> *We're Granite — a self-healing multi-agent AI system for semantic error diagnosis, verified repair, and recovery. When a multi-agent AI system fails — including when the wrong answer looks completely right — no published method can reliably say which agent caused it, and none can verify a fix before trusting it. That's the problem we're solving. I'm Muhammad Noor, with Shahoud Shahid and Saif Shahzad, supervised by Dr. Adil Majeed.*

Slide 2 — Problem & Users (~45s)

> *Who has this problem? Primarily engineering teams running multi-agent AI systems in or near production, plus downstream product teams and auditors who need traceable evidence. Today, without Granite, the workflow is: capture a trace with tools like LangSmith or Langfuse, have a human guess the root cause with no published accuracy, no check for silent failures, and no way to test a fix before shipping it. The field baseline is just 14.2% step accuracy. The consequence: the system ships a confident, wrong answer — no crash, no error, no signal anything went wrong. This maps to SDG 9 — reliability tooling as a new industrial category.*

Slide 3 — Existing Solutions & Prior FYP Comparison (~60s)

> *We checked the existing landscape structurally. ErrorProbe, the best published method, gets 59.5% attribution accuracy but admits it can't detect silent failures. Commercial platforms — LangSmith Engine, Arize Alyx, Langfuse, Braintrust — do trace capture and evaluation, but none verify a fix actually works, and most are locked to one stack. We also checked all 660 titles across 12 cohorts since 2019 for a prior departmental FYP — the closest is S24-011, hallucination detection in a single LLM's text, which covers roughly one of our four pipeline stages. The real gap: no system combines attribution, silent-failure detection, verified repair, and cross-architecture generalization in one pipeline. That's what Granite does.*

Slide 4 — Research & Technical Basis (~45s)

> *Four decisions the literature actually changed. We chose ErrorProbe as our baseline on performance, not recency. We committed to CrewAI and LangGraph — frameworks none of the nine papers we reviewed used. We chose constraint-based diagnosis over an LLM-judge, because LLM-judge approaches only reach 0.65 AUROC — barely above chance. And we rejected narrowing to a single framework, per the market survey's own advice. Across the published lineage, step-level accuracy climbs from 14.2% to just 59.5% — still wrong 40.5 points of the time, and none of these methods can see silent failures at all.*

Slide 5 — Complex Computing Challenge (~55s)

> *Five characteristics make this a genuine complex computing challenge, not a course project. Conflicting requirements — depth versus latency, privacy versus completeness. No obvious solution — the best published result is 59.5% after 15 months and 7 papers. Depth of knowledge — graph modeling, causal reasoning, tracing. Beyond standard practice — no framework does attribution, detection, and recovery together. And an ill-defined cause — three unknowns stacked: whether a failure happened, which step caused it, and what the correct output should have been.*

Slide 6 — Proposed Solution & Contribution (~75s)

> *Here's what we're building. The best method is 59.5% but untested on unseen architectures and misses silent failures — we build a cross-architecture benchmark and a diagnosis pipeline with constraint-based silent-failure verification, as a framework-agnostic SDK. Walking the diagram: a target CrewAI or LangGraph system feeds a trace in. Lane A adapts the framework and injects labeled faults into a benchmark store. Lane B normalizes the trace, builds an execution graph, and runs the Semantic Error Diagnostic and Backward Causal Tracer in parallel — unconditionally, not gated on an anomaly. Lane C proposes a repair, verifies it under controlled replay, and serves a dashboard through FastAPI. The LLM API sits outside the boundary on purpose — only 15 to 20% of components ever call it. Tier 2 — Recovery Controller and Verified Repair Memory — is explicitly gated on Mid-1 results, not claimed as delivered yet.*

Slide 7 — POC / Feasibility Evidence (~80s)

> *This is proof it works, not just an idea. 71.4% agent accuracy and 13.2% step accuracy across 5 real runs — that's a simplified, single-stage reproduction, not directly comparable to ErrorProbe's full 59.5%. Four of six repair attempts succeeded, both failures traced to one fixable cause. And 8 out of 8 spikes have real, recomputable evidence — including an honest negative: our preventive classifier scored near-random, 0.25, and we're reporting that, not hiding it. The headline result is spike 5 — counterfactual replay recovered a corrupted value, 97.2, to the correct 108.0, under a controlled replay protocol. [Gesture at the screenshot] And here's the live, clickable prototype behind these numbers.*

Slide 8 — Evaluation Plan & Risks (~55s)

> *Success and failure are defined before any result exists. We pre-registered an acceptance band — within 10 points of ErrorProbe's 59.5%. We measure agent and step accuracy, silent-failure precision and recall with false-positive rate on clean controls, and latency overhead under 50%. Success bands range from Acceptable — parity on Who&When — up to Exceptional — over 50% verified-repair recovery. Every top risk already has a stated fallback: missing the acceptance band means we report both numbers, not hide one; if the Tier-2 gates fail at Mid-1, Tier 1 alone still delivers the core contribution. And nothing we claim sits in core-replacement — the LLM is core-assist only.*

Slide 9 — Work Division & Iteration Plan (~45s)

> *Scope is split three ways, each with a stated "if removed" consequence. Shahoud owns Data and Benchmark — framework instrumentation and fault injection; without it, there's no cross-architecture benchmark to test against. I own Diagnosis — the graph, the ErrorProbe reproduction, and the Semantic Error Diagnostic; without it, there's no attribution and no contribution. Saif owns Platform — replay, the SDK, FastAPI, Postgres, deployment; without it, nothing is installable outside our machines. Five milestones carry us from scope-locked through full evaluation across FYP-1 and FYP-2.*

Slide 10 — DEI, Ethics & Legal (~25s)

> *Three commitments the panel can hold us to. DEI: open-source release, accessible color-and-text design, English-only claims stated honestly. Ethical: every repair is human-gated, advisory only, and negative results — like Spike 7 — are published, not buried. Legal: every dataset we use is MIT-licensed and verified against its actual LICENSE file, and AI usage is logged and disclosed every session.*

Delivery notes

- **Practice with a timer once** before assuming 8.5 min holds — reading speed varies person to person; adjust the 3 longest slides (6, 7, 3) first if you're over.
- **Slides 6 and 7 carry the most weight** — architecture and evidence are what a panel probes hardest. If you're short on time elsewhere, don't cut these; trim slide 4 or 10 instead.
- For deeper Q&A beyond this script, use `CCP_and_Spikes_Full_Reasoning.md` and `Literature_Review_Panel_Narrative.md` in this same folder.

Slides 5, 6, 7 — Expanded Speaking Notes

How to use this file: each slide below has your **original script** (unchanged, exactly as it was in the deck's speaker notes) followed by **added coverage** — the extra points needed so every visible box, lane, and number on the slide has something you can say about it. Read the original first; the additions slot in around it, they don't replace it.

Slide 5 — Complex Computing Challenge

Original script

> *This is the slide the panel uses to decide whether the project deserves capstone status. Concede up front that a trace-capture-plus-LLM-guess wrapper is a weekend's work, then name precisely what such a version would get wrong: no benchmark on unseen architectures, no way to tell a silent failure from a correct answer, no measured false-positive rate on clean controls, no evidence its fix works.*

Update: the diagram is gone, and the grid now shows only the **5 strongest characteristics**, renumbered 1–5 with no gaps, laid out as full-width elegant rows (same style as slide 10). Badge 5 (Ill-defined cause) has three stacked unknowns — read its resolution aloud if pressed: whether/which/what's-correct, each resolved by a distinct pipeline stage (see `CCP_and_Spikes_Full_Reasoning.md`).

Added coverage (all 5 boxes, badge order as on the slide)

- **Badge 1 — Conflicting requirements:** depth vs. latency is deeper graph/causal reasoning costing time a real-time repair loop can't always afford; privacy vs. completeness is capturing enough trace detail to diagnose correctly while not persisting data the operator doesn't want logged. Both are named trade-offs with a stated resolution, not hand-waved.
- **Badge 2 — No obvious solution:** say the number aloud — 59.5% is the best published step accuracy after 15 months and 7 papers of dedicated research. That absence of a known recipe is the whole justification for research-level effort.
- **Badge 3 — Depth of knowledge:** the diagnosis engine sits on graph algorithms (execution-graph construction), causal inference (backward causal tracing), and distributed-tracing concepts — not single-course material.
- **Badge 4 — Beyond standard practice:** no framework reviewed — academic or the 7 commercial platforms checked — combines attribution, general silent-failure detection, and verified repair together.
- **Badge 5 — Ill-defined cause:** covered above.

Held back (not on-slide): Unfamiliar issues, Diverse stakeholders, Significant consequences, Interdependence — none are weaker, they just aren't the 5 printed. Cover any of them only if asked; ready answers for each are in `CCP_and_Spikes_Full_Reasoning.md`.

Full reasoning and likely panel questions for all 9 (5 on-slide + 4 held back) are in `CCP_and_Spikes_Full_Reasoning.md` below.

Slide 6 — Proposed Solution & Contribution

Original script

> *Say unprompted: 'controlled, not deterministic' — the replay protocol is deterministic, LLM-step variance is measured over N replays, never assumed away. Say unprompted: the Semantic Error*

> *Diagnostic abstains where no checkable constraint exists, and the abstention rate is reported as a metric, not hidden. If asked about scope: in scope is 10 committed mechanisms; out of scope is message queues, Kubernetes, full RBAC, DR infra, multi-tenant billing; deployment target is Docker + a pip-installable SDK, operable by someone who didn't build it.*

Added coverage (walk the diagram once, lane by lane, before questions)

- **Actors:** the Target System (CrewAI/LangGraph workflow) feeds a raw execution trace in; the Operator installs the SDK against it and later views the dashboard.
- **Lane A — Data/Benchmark:** Framework Adapters normalize CrewAI/LangGraph hooks → Fault Injection Operators inject labeled faults deterministically at 0% API cost → Benchmark Store persists traces plus ground truth.
- **Lane B — Diagnosis:** Trace Normalizer standardizes the schema → Execution Graph Builder turns it into a graph at 0% API → Semantic Error Diagnostic and Backward Causal Tracer run **in parallel, unconditionally** (not gated on an anomaly — this is the headline contribution).
- **Lane C — Platform:** Repair Agent proposes one atomic fix → Verified Automatic Repair checks it under controlled replay → FastAPI Backend serves the Dashboard.
- **LLM API box:** sits *outside* the system boundary on purpose — only ~15-20% of components ever call it, everything else is deterministic. That's the "core-assist" claim, made visible, not just asserted.
- **Tier 2 note** (Recovery Controller + Verified Repair Memory): explicitly gated on Mid-1 criteria, not claimed as delivered. Say so if the panel points at it.

Slide 7 — POC / Feasibility Evidence

Original script

> *Lead with Spike 7's negative result and Spike 8's failure analysis — quote the guide's own words directly: a POC that disproved an assumption is more valuable than one that confirmed it. Every number on this slide is real and recomputable from files in the repository; offer to open one. In progress: full 3-stage reproduction, top FYP-1 priority. Planned: Recovery Controller + Verified Repair Memory Tier-2 gates, pre-registered Mid-1 criteria, not yet run. Prototype: 5 screens, mocked backend, dist-static/index.html.*

Added coverage

- **The 4 headline numbers:** 71.4% agent accuracy and 13.2% step accuracy are our **simplified single-stage** reproduction over 5 runs — not directly comparable to ErrorProbe's full 3-stage 59.5%, say so unprompted before a panelist notices the gap themselves. 4/6 is the Repair Agent's real success rate, both failures traced to one fixable cause. 8/8 means every spike has real, recomputable evidence, including the honest negative.

Spikes 1–6, one breath each, if asked:

- Spike 1 — ErrorProbe reproduction confirms the mechanism direction even simplified.
- Spike 2 — CrewAI instrumentation proves a real 3-node execution graph was captured.
- Spike 3 — Fault injection landed a labeled fault, with one self-correction kept as a finding, not hidden.
- Spike 4 / 4b — Semantic Error Diagnostic. Say it in two halves: the **seeded** mechanism test caught the $20.00-vs-$19.75 near-miss and every other known-wrong value with zero false

positives; the **live n=3 run** produced zero natural silent failures, so recall in the wild is unmeasured, not high.

- Spike 5 — Counterfactual replay is the **core novelty claim** — a corrupted 97.2 recovered to 108.0 under controlled replay.
- Spike 6 — Cyclic graph handling hit its 10,000-step safety cap as designed, proving the failure mode is contained, not silent.

Update: the Operator Workflow diagram was removed from this slide (it repeated logic already on slide 5) — the prototype screenshot now spans the full slide width instead. If asked about the human-approval workflow, it's a verbal answer: install, review the diagnosis before anything is applied, approve or reject, then a verified replay confirms the fix actually worked. Nothing here is autonomous.

CCP, Spikes & Contribution — Full Panel-Defense Reasoning

This is the deep version behind slides 5, 6 and 7 — not what's on the slide, but the reasoning underneath it, written so you can answer a follow-up question the slide itself doesn't spell out. Organized as: statement → why it's true → likely panel question → model answer.

Part 1 — CCP characteristics (5 on-slide + 4 held back)

Numbering note: headings below follow the on-slide badge order (1–5) exactly as it now appears on slide 5 — the 5 strongest, most classic CCP arguments. Badges 1–5 line up with real characteristics 1, 2, 3, 5, 9 respectively (noted per section). The other 4 real characteristics (4, 6, 7, 8) are held back in full below the on-slide five — not weaker arguments, just not the ones printed, so ready if a panelist asks about any of them.

1. Conflicting requirements

Why it's true: Two real trade-offs, not one. (a) Depth vs. latency — deeper causal/graph reasoning gives better attribution but costs time a production repair loop can't always spend. (b) Privacy vs. completeness — diagnosing well needs a rich trace, but the operator may not want every field of that trace persisted or logged.

Likely question: *"How do you actually resolve privacy vs. completeness — isn't that just a trade-off you're leaving unsolved?"*

Model answer: "We don't claim to resolve it universally — we make it a configurable boundary, not a hidden default. The SDK captures what's needed for attribution (agent id, step, inputs/outputs relevant to the constraint being checked), and what gets persisted to the Benchmark Store vs. discarded after diagnosis is an operator-set policy, not baked into the pipeline. The trade-off is real and stays visible — that's the design decision, not a silent compromise."

Likely question: *"And depth vs. latency — what's your actual answer?"*

Model answer: "Our NFR target is under 50% latency overhead versus the uninstrumented baseline. Where deeper causal tracing would blow that budget, the Semantic Error Diagnostic is deliberately the fast, 0%-API, deterministic path that runs unconditionally — the heavier LLM-based verification (Investigator stage) only runs when the cheap check can't resolve it. That's the actual trade-off resolution: cheap-first, escalate-only-when-needed."

---

2. No obvious solution

Why it's true: Best published step-level accuracy is 59.5% (ErrorProbe), after 15 months and 7 papers across ICML/ICLR/AAAI/ACL. If there were an obvious solution, someone in that lineage would already have it.

Likely question: *"Why should a proposal-stage FYP succeed where 7 papers didn't?"*

Model answer: "We're not claiming to out-attribute ErrorProbe — we adopt its algorithm as our baseline. The contribution is the three things none of those 7 papers do together: general silent-failure detection (not anomaly-gated), verified counterfactual repair, and cross-architecture generalization to frameworks none of them tested. We're not solving attribution better; we're solving the combination none of them attempted."

---

3. Depth of knowledge

Why it's true: Requires graph modeling (execution-graph construction), causal reasoning (backward causal tracing to localize root cause), and distributed-systems tracing concepts — a genuine cross-topic synthesis, not one course's syllabus.

Likely question: *"Isn't this just calling an LLM API — where's the depth?"*

Model answer: "The LLM is core-assist only — 15-20% of components. The actual attribution logic (Culprit Agent, Error Step Localization) runs on graph features at 0% API cost. The depth is in the graph construction, the causal-tracing algorithm, and the constraint-verification logic in the Semantic Error Diagnostic — none of that is an LLM call."

---

4. Beyond standard practice (real characteristic 5)

Why it's true: No existing framework — academic or the 7 commercial platforms checked (LangSmith, Langfuse, Braintrust, Arize, Maxim, Opik, DeepEval) — combines step-level attribution, general silent-failure detection, and verified counterfactual repair in one evaluated pipeline.

Likely question: *"Arize/LangSmith already do RCA — what's actually new?"*

Model answer: "They do parts of this — trace capture, evaluation, some RCA. None of them, per our own literature and market check, do all three together: attribution + general silent-failure detection (not just RAG-groundedness) + verified repair with a measured recovery rate. That combination, evaluated as one pipeline, is the gap — not any single piece of it."

---

5. Ill-defined cause (real characteristic 9)

Why it's true: Three unknowns stacked together — *whether* a failure happened at all (silent failures give no signal), *which* step caused it, and *what* the correct output should have been. None of the three is given.

Likely question: *"How can you claim to solve an 'ill-defined' problem — doesn't that mean it's unsolvable?"*

Model answer: "Ill-defined doesn't mean unsolvable — it means each unknown needs its own resolution, which is exactly the pipeline: the Semantic Error Diagnostic resolves *whether* (via constraint verification, abstaining when it can't tell — the abstention rate is reported, not hidden); the Backward Causal Tracer resolves *which* step; and the Repair Agent's proposed correction, verified by replay, resolves *what* the correct output should have been. Three unknowns, three distinct mechanisms, each measured separately."

Held back — 4 characteristics not on the slide

These are real, argued in `Granite_CCP_Justification_Seoul_Accord.docx`, and not weaker — they just aren't the 5 printed on slide 5. Cover any of them if asked directly.

Unfamiliar issues *(real characteristic 4)* — Delayed errors (a silent failure can surface steps later), non-determinism (identical inputs can produce different agent traces), and cross-architecture generalization (CrewAI and LangGraph are structurally different) — none of these are issues a standard course project has to confront.

If asked about non-determinism: "We don't assume it away — we measure it. The Verified Automatic Repair protocol is deterministic in its own steps (frozen upstream context, pinned decoding parameters),

and LLM-step variance is measured over N replays rather than a single point estimate — exactly what Spike 5 demonstrates."

Diverse stakeholders *(real characteristic 6 — the weakest of the 9)* — the other named stakeholders (engineering team, downstream product teams, auditors) are supported by literature + a 7-vendor survey, but live contact with a real stakeholder is only scheduled for Mid-2.

If asked: "It's real, just not shown — we don't have a working system to show a stakeholder yet. It's scheduled for Mid-2, when there's something real to react to."

Significant consequences *(real characteristic 7)* — a missed silent failure isn't a crash — it's a confidently wrong output a downstream system or person acts on, undetected.

If asked for a concrete example: "The near-miss from our mechanism test: a stated $20.00 against a true $19.75. In a real refund or billing pipeline, that's money moving on a wrong number with no crash, no exception, no log line flagging it."

If they press on where that number comes from — answer it straight: it is from the **seeded** mechanism test (`dev/spikes/poc/poc_spike4b_mechanism_output.json`), which feeds known-wrong values directly to the rule-based checker. It is **not** from the live n=3 run — that run produced zero natural silent failures, so recall in the wild is unmeasured. Say this before they find it; the distinction is the whole credibility of the claim.

Interdependence *(real characteristic 8 — the "dependency" question)* — agents in a workflow pass state to each other; an error in one agent's output can corrupt every downstream agent's reasoning.

If asked how a dependency is actually shown: "The execution graph is the proof — nodes are actions, edges are dependencies, built directly from the real trace, not inferred. The Backward Causal Tracer walks those edges backward from the visible symptom to find where the error originated. Spike 2 demonstrates this concretely: a real 3-node graph with 2 clean edges captured from an actual CrewAI run."

Part 2 — All 8 spikes, full reasoning

#	Spike	Why it matters beyond the one-line result
1	ErrorProbe reproduction (simplified)	Proves the mechanism *direction* is right even at 1-stage instead of ErrorProbe's full 3-stage — the range (62.5–85.7% agent, 0–25% step) across 5 real runs is reported honestly as a range, not cherry-picked.
2	CrewAI instrumentation	Proves trace capture actually works on a real framework, not a mock — 3 real nodes, 2 clean edges, from an actual run.
3/3b	Fault injection	Proves we can deterministically inject a labeled fault — the ground truth *is* the injection operator, so no manual annotation cost (this is the AgenTracer idea we reuse). One case self-corrected; kept and reported as a finding, not hidden, because it's evidence the system doesn't always behave as expected and we say so.
4 / 4b	Semantic Error Diagnostic	The headline mechanism, and the one to state in two halves. **Live n=3 run (spike 4):** the model got all three tasks right, so there were zero natural silent failures — `detector_recall_on_silent_failures: null`, unmeasured, with zero false positives. **Seeded mechanism test (spike 4b):** 3 of 3 known-wrong values caught, including the $20.00-vs-$19.75 near-miss that a "close enough" check would wave through, and 0 of 3 correct values flagged — including one differing only by legitimate rounding. The mechanism works; its hit rate on natural failures does not yet have a number. Volunteering that split is stronger than being caught by it.
5	Controlled counterfactual replay	The **core novelty claim** — a corrupted value (97.2) recovered to the correct value (108.0) under a controlled replay protocol, with `recovery_success = true` recomputable from the repo. This is the one no other reviewed method demonstrates.

6	Cyclic graph handling	Proves failure containment, not silent hangs — disabling cyclic support correctly hits the 10,000-step safety cap rather than looping forever undetected.
7	Preventive classifier	The **honest negative** — precision/recall/F1 = 0.25, near-random. Reported because the handbook's own guidance says a POC that disproves an assumption is more valuable than one that confirms it. This is the strongest credibility signal on the whole slide — it shows we test our own assumptions and report failure.
8	Repair Agent	4/6 real success rate; both failures traced to one shared, fixable cause (not two unrelated bugs) — proves the failure mode is understood, not mysterious.

If asked "why include a failed spike (7) at all": quote the guide directly — a POC that disproves an assumption is more valuable than one that confirms it. Spike 7 tells us *not* to rely on a preventive classifier and to invest in the reactive Semantic Error Diagnostic instead — that's a real design decision the negative result produced.

Part 3 — Proposed Solution & Contribution (slide 6) reasoning

Why ErrorProbe as the baseline, not the newest paper: chosen on performance (59.5%, beats both its ancestors on their own benchmarks), not recency — it isn't even the newest of the four ACL 2026 papers reviewed.

Why CrewAI + LangGraph specifically: neither framework was used by any of the 9 papers reviewed. That's the actual generalization test — none of the literature tests transfer to an architecture unseen during benchmark construction; committing to two frameworks nobody else tested is the point, not a convenience choice.

Why constraint-based Semantic Error Diagnostic, not an LLM-judge: literature shows LLM-judge approaches cap out at AUROC ≤ 0.65 — barely above chance. A deterministic, rule/schema-based check is both cheaper (0% API) and more reliable for the specific failure mode (semantically wrong but structurally valid output) than another LLM judging an LLM.

Why the LLM is core-assist only (15-20%): the swap test — if the LLM API were replaced, the contribution (graph-based attribution, constraint verification, replay-based repair verification) survives. Only the Investigator-stage verification and repair-candidate generation actually call it.

Why Docker + pip-installable SDK as the deployment target: operability by someone who didn't build it is the actual bar — a system only the authors can run isn't a deliverable, it's a demo.

Why Tier 2 (Recovery Controller / Verified Repair Memory) is explicitly gated: these depend on Mid-1 reproduction results; committing to them unconditionally before that gate would be promising something not yet earned. The gate structure is the honest way to commit scope without overpromising.

Literature Review — Workflow & Panel Narrative

This is grounded directly in `markdowns/03-market-and-literature-research/Literature_Review_Sheet.xlsx` (6 sheets: Start Here, Papers, The Numbers, Datasets, The Gap, Lineage) — nothing here is invented; every figure traces back to a paper's own results table or a verified download. Use this to explain *how* the review was done and to argue convincingly that Granite is grounded in, not disconnected from, related work.

---

1. The workflow — how the review was actually done

Step 1 — Find the field. Searched for automated failure attribution in multi-agent AI systems. Found the field is genuinely new: it barely existed before April 2025, when Who&When (ICML 2025) formalized it as a research problem and built the first human-annotated benchmark.

Step 2 — Read full text, not abstracts. All 7 papers were read end-to-end. The "own admitted gap" recorded for each paper is a direct quote from that paper's own Limitations section — this is what makes it defensible to cite a weakness *against* a paper: it's not our interpretation, it's their own admission.

Step 3 — Track lineage, not just topic. Rather than treating all 7 papers as one undifferentiated pile, citation links were traced directly (Sheet 6). This revealed **two separate clusters**, not one crowded field: a main lineage descending from Who&When (AgenTracer, SDBL, TraceElephant, ErrorProbe), and two independent papers (AgentAsk, PROTEA) with no citation or benchmark overlap with that lineage at all.

Step 4 — Pick the baseline on performance, not recency. ErrorProbe was chosen because it beats both its lineage ancestors (Who&When, AgenTracer) on their own benchmarks — not because it's the newest of the four ACL 2026 papers in the set.

Step 5 — Verify every number and every license at the source, not at the citation. Two honesty rules were applied throughout:

- *Verify at the source:* TracerTraj was described as Apache 2.0 in ErrorProbe's own paper — direct repository inspection found **no LICENSE file at all** and no released dataset. That single check is now the standing lesson behind every other license claim in the sheet: a citation is not a license check.
- *Report ranges as ranges:* AgenTracer publishes ~37–43%, not a single number — it's reported as a range everywhere in the sheet, the same discipline applied to our own Spike 1 results.

Step 6 — Check the department, not just the wider field. All 660 registered titles across 12 published cohorts (Spring 2019 – Fall 2025) were checked structurally, not just by title, for a prior FYP covering the same ground. Closest match: S24-011-D-HDLM (hallucination detection in one LLM's text) — confirmed as covering roughly one of Granite's four pipeline stages, not the whole problem.

Step 7 — Turn the reading into a gap statement, not just a summary. Sheet 5 ("The Gap") is the actual argument the whole Stream A claim rests on: five specific, evidenced gaps, each paired with what Granite does about it.

---

2. The numbers — and the caveat that makes them defensible

Benchmark / metric	Who&When	AgenTracer	SDBL	ErrorProbe (our baseline)
Who&When-Algo — step accuracy	14.2%	—	39.68%	**59.50%**
Who&When-Hand — step accuracy	14.2%	—	27.78%	31.00%
TracerTraj — step accuracy	—	~37–43%	—	44.40%

Who&When-Algo — agent accuracy	53.5%	—	63.49%	60.30%

The caveat, say it before a panelist says it to you: this is a literature-based comparison, not a controlled experiment. Every paper chose its own backbone LLM (SDBL: GPT-4o/DeepSeek; AgenTracer: its own trained 8B; ErrorProbe: Claude 3.7 Sonnet, GPT-OSS-120B, Qwen3-32B). No paper runs all methods on an identical backbone, so part of ErrorProbe's margin could reflect a stronger backbone rather than a better method. Granite controls for this *inside its own experiments* and dual-reports the result rather than presenting a single number as clean.

Our own POC numbers, for honest comparison (Spike 1, 5 runs, local 3B model):

- Agent accuracy: 71.4% mean (62.5–85.7% range) vs. ErrorProbe's published 60.30% — but on a **simplified, single-stage** pipeline, not a like-for-like claim.
- Step accuracy: 13.2% mean (0–25% range) vs. ErrorProbe's published 59.50% — far below, and expected, since this is a 1-stage reproduction of ErrorProbe's full 3-stage method.
- Acceptance band: ±10 points of 59.5%, pre-registered in the plan before any run — landing outside it is dual-reported, not quietly revised after the fact.

---

3. Datasets — role, verification, and what got cut

Dataset	Role	License (verified)	Status
Who&When	Baseline reproduction	MIT	ACTIVE — load-bearing
tau-bench (retail)	Injection substrate	MIT	ACTIVE
GSM8K	Injection substrate	MIT	ACTIVE
TracerTraj-2.5K	Would have been baseline data	**No LICENSE file exists**	BLOCKED
MATH	Injection substrate	MIT	BENCHED (numeric format fits GSM8K better)
MBPP+	Injection substrate	Permissive	BENCHED
GAIA	Injection substrate	Gated, no redistribution	BENCHED, must be purged from history before going public
Our own benchmark	The contribution	MIT/CC BY 4.0 (target)	TO BUILD — 500–1,000 labeled traces, ground truth by construction

The one line that shows real diligence: TracerTraj was blocked *after* being planned as baseline data, because direct inspection (not the paper's own description) found no license file and no released dataset. That's the kind of check a panel wants to see happened.

---

4. The gap — the actual argument, in five rows

1. **Silent failures are undetectable by every method reviewed** — ErrorProbe's own Limitations section admits its pipeline is anomaly-gated, so an error producing no anomaly is structurally invisible to it. Every other method in the lineage inherits the same gating assumption. → Granite's Semantic Error Diagnostic runs unconditionally, in parallel, never gated on an anomaly, and reports its own abstention rate as a metric.

2. **No paper tests transfer to an unseen architecture** — confirmed after full-text reading of all seven; ErrorProbe's "cross-domain transfer" is transfer across *task* domains within its own tested ecosystem, not architecture transfer. → Granite commits to CrewAI and native LangGraph, neither used by any of the nine papers reviewed.

3. **No method verifies a fix before trusting it** — PROTEA proposes fixes but requires human approval with no automated confidence scoring; nothing else reviewed measures whether a repair actually works before applying it. → Verified Automatic Repair measures a recovery rate over N controlled replays (Spike 5: 97.2 → 108.0).

4. **Nobody measures the false-positive rate** — absent from every paper and all seven commercial platforms checked. → Granite's benchmark includes 15–20% deliberately clean controls, provisional target under 5% FPR.

5. **No prior departmental FYP covers this** — checked against all 660 titles; S24-011-D-HDLM is the closest, covering roughly one of Granite's four stages.

The one-sentence version to close with: *"No system checked — academic or commercial — confirms step-level attribution accuracy together with general silent-failure detection, counterfactual fix verification, and cross-architecture generalization, evaluated as one pipeline. The gap is the combination, not any single missing feature."*

---

5. Convincing the panel this is genuinely related-work-grounded

If challenged with *"this looks like you're just picking holes in other people's papers"* — the answer is that every gap claimed is the paper's **own stated limitation**, not our interpretation, and every number is pulled from a results table or a verified download, never estimated. If challenged with *"the field looks crowded, why does this need to exist"* — point to Sheet 6: two separate, non-overlapping clusters, six independent follow-ups to a single 2025 paper in about 15 months, and step-level accuracy still stuck at 59.5% despite all of it. That pace and that ceiling, together, are the argument that the problem is real, active, and still open — not solved and not overcrowded.